KU LEUVEN

FACULTEIT INDUSTRIELE
INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Geavanceerde Computerarchitectuur

report 1
MEI-ecfs

Charlotte BERT
Sander HAUSTRAETE

Academiejaar 2025 - 2026

1 Context

Het doel van dit labo is het schrijven en uitvoeren van parallelle code op de GPU. De opdracht is het omkeren van een input-array. Hierbij wordt gebruikgemaakt van een grid-configuratie met meerdere blokken en threads. Het proces vereist correcte geheugenmigratie tussen host en device. Op het einde worden de resultaten geobserveerd.

2 Methodologie

We schreven een reverse-kernel om een array flip te realiseren. Binnen reverse kopiëren we de array in omgekeerde volgorde naar een andere array door de array-indices aflopend te doorlopen. We kozen ervoor om te kopiëren in plaats van te overschrijven, zodat we de input en het resultaat kunnen vergelijken (zie codefragment 1).

3 Observatie/ resultaat

```
Input: 0 1 2 3 4 5 6 7 8
Result: 8 7 6 5 4 3 2 1 0
Time taken: 10.9885 ms
```

Figuur 1: Terminal output

We printten zowel de input als het resultaat om een directe vergelijking te kunnen maken. Uit de output blijkt dat het resultaat effectief de omgekeerde array van de input is. Dit komt overeen met het verwachte en gewenste resultaat.

Na de code tien keer te hebben uitgevoerd, bekomen we een gemiddelde rekentijd van 5.36 ms.

4 Vragen

Wat is de grootst mogelijke blok grootte?
Het limiet van threads per blok is 1024 [1].

5 Conclusie

In dit labo hebben we succesvol parallelle GPU-code geïmplementeerd voor het omkeren van een array. De reverse-kernel functioneerde zoals verwacht en leverde de omgekeerde array op.

Referenties

[1] NVIDIA. *CUDA C++ Programming Guide*.

```cpp
#include <iostream>
#include <cuda_runtime.h>

// CUDA kernel reverse array (copy into other array in reverse order)
__global__ void reverse(int *a, int *b, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        b[idx] = a[n - 1 - idx];
    }
}

int main() {
    const int N = 10;

    int a[N], b[N]; // a:input, b:output
    int *d_a, *d_b;

    for (int i = 0; i < N; i++) {
        a[i] = i;
    }


    // Allocate device memory
    cudaMalloc((void**)&d_a, N * sizeof(int));
    cudaMalloc((void**)&d_b, N * sizeof(int));

    // Copy data from host to device
    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);

    // Timing
    cudaEvent_t start_cuda, stop_cuda;
    cudaEventCreate(&start_cuda);
    cudaEventCreate(&stop_cuda);
    cudaEventRecord(start_cuda);

    // Launch kernel (1 block of N threads)
    reverse<<<1, N>>>(d_a, d_b, N);

    // stop timing
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda);
    cudaEventSynchronize(stop_cuda);
    float ms;
    cudaEventElapsedTime(&ms, start_cuda, stop_cuda);

    // Copy result back to host
    cudaMemcpy(b, d_b, N * sizeof(int), cudaMemcpyDeviceToHost);
    cudaMemcpy(a, d_a, N * sizeof(int), cudaMemcpyDeviceToHost);
    // Print results
    std::cout << "Input: ";
```

```
51      for (int i = 0; i < N; i++) {
52          std::cout << a[i] << " ";
53      }
54      std::cout << std::endl;
55
56      std::cout << "Result: ";
57      for (int i = 0; i < N; i++) {
58          std::cout << b[i] << " ";
59      }
60      std::cout << std::endl;
61
62      std::cout << "Time taken: " << ms << " ms" << std::endl;
63
64      // Free device memory
65      cudaFree(d_a);
66      cudaFree(d_b);
67
68      return 0;
69  }
```

Codefragment 1: Code labo 1

FACULTEIT INDUSTRIELE INGENIEURSWETENSCHAPPEN
TECHNOLOGIECAMPUS GENT
Gebroeders De Smetstraat 1
9000 GENT, België
tel. + 32 92 65 86 10
fax + 32 92 25 62 69
iiw.gent@kuleuven.be
www.iiw.kuleuven.be

LID VAN ASSOCIATIE KU LEUVEN